

# TP2 — Terraform sur Azure

Infrastructure as Code — Cas fil rouge ShopEasy

---

|           |                                          |
|-----------|------------------------------------------|
| Auteurs   | Olivier Falahi & Paul Claverie           |
| Formation | Mastère Dev Manager Full Stack — EFREI   |
| Module    | Optimisation du SI par l'apport du Cloud |
| Outil     | Terraform · Provider azurearm            |
| Cloud     | Microsoft Azure (swedencentral)          |
| Année     | 2025 / 2026                              |

Projet Terraform, réponses aux ateliers, analyses FinOps & sécurité, note technique et code source.

# Sommaire et conformité au rendu

Ce document unique regroupe l'ensemble du rendu du TP2. Il répond aux **livrables attendus** (section 20 du sujet).

| Livrable attendu                                    | Traité dans                             |
|-----------------------------------------------------|-----------------------------------------|
| 1. Arborescence complète du projet Terraform        | Annexe A                                |
| 2. Les fichiers .tf                                 | Annexe B (code source intégral)         |
| 3. Captures init / validate / plan / apply / output | Annexe C — Preuves d'exécution          |
| 4. Capture du portail Azure (ressources)            | Annexe C — Preuves d'exécution          |
| 5. Capture de la page web via le Load Balancer      | Annexe C — Preuves d'exécution          |
| 6. Analyse de dérive (drift)                        | Compte rendu — Atelier 11               |
| 7. Tableaux FinOps et sécurité complétés            | Analyse FinOps & sécurité               |
| 8. Note technique courte                            | Note technique (en tête de ce document) |

Les captures d'écran sont regroupées en **Annexe C** et référencées par nom de fichier dans la checklist technique du compte rendu.

## Contenu

1. Note technique (synthèse des choix + architecture)
2. Compte rendu des ateliers (1 à 14) + checklist + analyse de dérive
3. Analyse coût (FinOps), sécurité et maintenabilité
4. Mise en autonomie — Option A : subnet privé pour les données
5. Quiz de validation (20 questions)
6. Annexe A — Arborescence du projet
7. Annexe B — Code source Terraform
8. Annexe C — Preuves d'exécution (captures)

# TP2 Terraform — Note technique

Synthèse des choix d'industrialisation de l'infrastructure ShopEasy avec Terraform sur Azure.

## 1. Objectif

Transformer l'architecture conçue au TP1 (réalisée manuellement via le portail) en **infrastructure déclarative**, reproductible, versionnée et destructible. Le code Terraform devient la source de vérité de l'environnement de développement ShopEasy, et la base des futurs environnements de recette et de production.

## 2. Architecture déployée

```
graph TD
    Internet([Internet])
    LB[Load Balancer Standard\npip-shopeasy-dev-lb]
    subgraph vnet [VNet shopeasy-dev 10.20.0.0/16]
        subgraph snetWeb [snet-web 10.20.1.0/24]
            VM1[vm-shopeasy-dev-web-1\nNginx (cloud-init)]
            VM2[vm-shopeasy-dev-web-2\nNginx (cloud-init)]
        end
        subgraph snetData [snet-data 10.20.2.0/24 prive]
            DB[Future base de donnees]
        end
    end
    Storage[Storage Account\ncontainer documents (prive, versioning)]
    Internet -->|HTTP 80| LB
    LB --> VM1
    LB --> VM2
    VM1 -->|SQL 1433| snetData
    VM2 -->|SQL 1433| snetData
    VM1 --> Storage
    VM2 --> Storage
```

Schéma d'architecture (notation Mermaid).

Composants : 1 Resource Group, 1 VNet, 2 subnets (web + data privé), 2 NSG, 2 VM Linux Ubuntu, 3 IP publiques, 1 Load Balancer, 1 Storage Account + container privé.

## 3. Choix techniques

| Choix                                       | Justification                                                                      |
|---------------------------------------------|------------------------------------------------------------------------------------|
| <b>Provider</b> <code>azurerm</code> ~> 4.0 | Standard Azure, lisible, documenté ; <code>random</code> pour l'unicité du Storage |
|                                             | Lisibilité, revue de code, limitation des conflits Git                             |

| Choix                                                                                         | Justification                                                                      |
|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>Découpage par responsabilité</b> ( network / security / compute / loadbalancer / storage ) |                                                                                    |
| <b>Variables + locals</b>                                                                     | Aucune valeur en dur ; réutilisable multi-environnement ; tags centralisés         |
| <b>count = 2</b> sur VM/NIC/IP                                                                | Redondance web sans duplication de code                                            |
| <b>cloud-init via templatefile</b>                                                            | Provisioning automatique de Nginx, page personnalisée par serveur                  |
| <b>Load Balancer Standard + probe</b>                                                         | Suppression du SPOF web, bascule automatique sur VM saine                          |
| <b>Storage container privé + versioning + TLS 1.2</b>                                         | Confidentialité des documents métier, protection contre suppression accidentelle   |
| <b>Tags common_tags</b>                                                                       | Gouvernance et FinOps ( project , environment , owner , managed_by , cost_center ) |
| <b>Subnet snet-data privé (option A)</b>                                                      | Segmentation réseau, préparation d'une couche données isolée                       |

## 4. Conventions de nommage

<type>-<projet>-<environnement>[-<role>][-<index>] → rg-shopeasy-dev , vnet-shopeasy-dev , snet-web , nsg-shopeasy-dev-web , vm-shopeasy-dev-web-1 , pip-shopeasy-dev-lb . Le Storage Account suit la contrainte Azure (minuscules/chiffres, unicité globale) : shopeasydevdocs<suffixe-aléatoire> .

## 5. Adaptations liées à la souscription Azure for Students

| Contrainte | Valeur sujet  | Valeur retenue   | Raison                                          |
|------------|---------------|------------------|-------------------------------------------------|
| Région     | francecentral | swedencentral    | Policy Students interdisant francecentral       |
| Gabarit VM | Standard_B1s  | Standard_B2ts_v2 | B1s indisponible (capacité) sur la souscription |

Ces valeurs sont **paramétrées** (variables location , vm\_size ) : le passage à francecentral / B1s sur une souscription standard ne nécessite qu'un changement de terraform.tfvars .

## 6. Sécurité et gestion des secrets

- Aucun secret dans le code : seule la **clé SSH publique** est référencée par chemin ( file(var.ssh\_public\_key\_path) ).
- terraform.tfvars et le **state** sont ignorés par Git ( .gitignore ).
- SSH restreint à allowed\_ssh\_cidr ; Storage privé ; subnet data sans IP publique.

- En production : backend `azurerm` distant chiffré (RBAC), Azure Key Vault, Bastion, Application Gateway + WAF.

## 7. Cycle de vie et reproductibilité

---

```
terraform init      # telecharge providers
terraform fmt       # formate
terraform validate  # verifie
terraform plan      # previsualise
terraform apply     # deploie
terraform output    # IP LB, RG, Storage
terraform destroy   # nettoie (obligatoire en fin de seance)
```

## 8. Limites et évolutions

---

Environnement volontairement minimal (pédagogique). Évolutions vers la production : suppression des IP publiques des VM + Bastion, state distant, Application Gateway + WAF, Azure SQL managé avec private endpoint dans `snet-data`, observabilité (Azure Monitor / Log Analytics), modularisation Terraform et pipeline CI/CD ( `fmt` / `validate` / `plan` / `tfsec` / `infracost` / `approbation` ).

# TP2 Terraform — ShopEasy — Compte rendu des ateliers

---

Cas fil rouge : industrialisation de l'infrastructure **ShopEasy** sur Azure avec Terraform. Région : `swedencentral` — Resource Group : `rg-shopeasy-dev` — VNet : `10.20.0.0/16`.

**Note de contexte.** Comme au TP1, l'abonnement Azure for Students impose une policy limitant les régions ( `francecentral` interdite ) : la région retenue est `swedencentral`. Le gabarit `Standard_B1s` étant indisponible (capacité), les VM utilisent `Standard_B2ts_v2` (paramétré via la variable `vm_size`).

**État du déploiement.** Infrastructure **déployée et validée** le 25/06/2026 sur la souscription Azure for Students (région `swedencentral`). terraform apply a créé **24 ressources** sans erreur. Valeurs réelles issues de terraform output : Load Balancer `20.91.219.236`, VM web-1 `4.223.225.195`, VM web-2 `4.223.111.127`, Storage `shopeasydevdocsbhvsip`. Les preuves d'exécution (terminal, portail, navigateur) sont regroupées en **Annexe C** du PDF et listées dans `06_captures_a_faire.md`.

---

## Atelier 1 — Initialiser le projet Terraform

Arborescence créée dans `tp2/terraform/` : `versions.tf`, `providers.tf`, `variables.tf`, `locals.tf`, `network.tf`, `security.tf`, `compute.tf`, `loadbalancer.tf`, `storage.tf`, `outputs.tf`, `terraform.tfvars.example`, `.gitignore` et `templates/cloud-init.yml`.

**Point de contrôle — pourquoi `terraform.tfstate` ne doit pas être publié dans un dépôt non sécurisé ?** Le state est la cartographie complète des ressources gérées par Terraform. Il contient des identifiants de ressources, des métadonnées et parfois des **valeurs sensibles en clair** (chaînes de connexion, clés, mots de passe générés, adresses IP, identifiants de comptes de stockage). Le publier reviendrait à exposer la topologie de l'infrastructure et des secrets exploitables par un attaquant. Il est donc exclu via `.gitignore` (et, en entreprise, stocké dans un backend distant chiffré et soumis au RBAC).

---

## Atelier 2 — Déclarer les providers

`versions.tf` épingle `terraform >= 1.6.0`, `azurerm ~> 4.0` et `random ~> 3.6`. `providers.tf` active le provider `azurerm` avec `features {}`.

**Point de contrôle — résultat attendu de `terraform validate` :**

```
Success! The configuration is valid.
```

L'épinglage des versions ( `required_version` , `version` ) garantit la reproductibilité : un collègue ou une pipeline CI/CD utilisera exactement les mêmes versions de providers, évitant les écarts de comportement.

## Atelier 3 — Paramétrer le projet

Variables d'entrée déclarées dans `variables.tf` ( `project` , `environment` , `location` , `vm_size` , `admin_username` , `ssh_public_key_path` , `allowed_ssh_cidr` , et les préfixes réseau). `locals.tf` calcule `prefix = "shopeasy-dev"` et le bloc `common_tags` .

Le secret SSH n'est jamais en clair : seul le **chemin** vers la clé publique est passé ( `ssh_public_key_path` ), et `terraform.tfvars` (qui contient les valeurs concrètes comme `allowed_ssh_cidr` ) est ignoré par Git. Un `terraform.tfvars.example` documente les valeurs attendues sans rien exposer.

## Atelier 4 — Groupe de ressources et réseau

Créés dans `network.tf` : `rg-shopeasy-dev` , `vnet-shopeasy-dev` ( `10.20.0.0/16` ), `snet-web` ( `10.20.1.0/24` ) et — au titre de la mise en autonomie (option A) — `snet-data` ( `10.20.2.0/24` ).

### Réponses aux questions (8.3) :

- Pourquoi une plage privée pour le VNet ?** Les plages `10.0.0.0/8` , `172.16.0.0/12` , `192.168.0.0/16` (RFC 1918) ne sont pas routables sur Internet. Elles évitent tout conflit d'adressage avec des réseaux publics, permettent de cloisonner les ressources et imposent de passer par des points d'entrée contrôlés (Load Balancer, IP publiques explicites) pour exposer un service. C'est la base de la segmentation et de la sécurité réseau.
- Que faudrait-il ajouter pour isoler une base de données dans un subnet dédié ?** Un **subnet dédié** ( `snet-data` , déjà ajouté en option A), un **NSG** restrictif n'autorisant que le port de la base (1433 pour SQL) **depuis le subnet web uniquement**, et idéalement un **private endpoint** vers le service managé (Azure SQL) plus une **delegation** de subnet le cas échéant. Aucune IP publique ne doit être attachée à ce subnet.
- Pourquoi séparer les fichiers Terraform au lieu de tout mettre dans `main.tf` ?** Lisibilité et maintenabilité : chaque fichier regroupe une responsabilité (réseau, sécurité, calcul, stockage). Cela facilite les revues de code, limite les conflits Git en équipe, et permet de retrouver rapidement une ressource. Terraform charge de toute façon **tous** les `.tf` du dossier, le découpage est purement organisationnel.

## Atelier 5 — Sécuriser le réseau avec un NSG

`security.tf` crée `nsg-shopeasy-dev-web` (Allow-HTTP 80 depuis Internet, Allow-SSH-Admin 22 depuis `allowed_ssh_cidr`) et son association au subnet web. L'option A ajoute `nsg-shopeasy-dev-data` (1433 depuis le subnet web + Deny-All entrant).

### Analyse de sécurité (9.2)

| Flux                                                     | Autorisé ?             | Justification                                                           | Risque résiduel                                                                                            |
|----------------------------------------------------------|------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Internet → HTTP (80)                                     | Oui                    | L'application web doit être joignable publiquement via le Load Balancer | Surface exposée au niveau applicatif (DDoS L7, failles web) → atténuée par WAF/Application Gateway en prod |
| SSH depuis votre IP (22, <code>allowed_ssh_cidr</code> ) | Oui                    | Administration ponctuelle restreinte à l'IP de l'apprenant              | IP dynamique pouvant changer ; usurpation si poste compromis → Bastion en prod                             |
| SSH depuis Internet complet (0.0.0.0/0)                  | <b>Non</b>             | Surface d'attaque massive, brute-force permanent                        | Aucun (règle volontairement absente)                                                                       |
| Tout trafic sortant                                      | Oui (par défaut Azure) | Permet <code>apt /cloud-init</code> de récupérer Nginx                  | Exfiltration possible si VM compromise → filtrage egress + firewall en prod                                |

**Recul (production).** L'accès SSH direct serait supprimé au profit d'**Azure Bastion**, d'un VPN ou d'un jump server, voire d'une administration sans accès direct (configuration uniquement par IaC + cloud-init).

## Atelier 6 — Deux machines virtuelles Linux

`compute.tf` crée, avec `count = 2` : 2 IP publiques `Standard`, 2 NIC dans `vnet-web`, 2 VM Ubuntu 22.04 (`vm_size = Standard_B2ts_v2`). Le `cloud-init.yml` installe Nginx et publie une page « ShopEasy - serveur web N » via `templatefile` (variable `server_index`).

### Réponses aux questions (10.4) :

1. **À quoi sert `count` ?** À créer plusieurs instances identiques d'une ressource à partir d'un seul bloc, indexées par `count.index`. Ici, 2 VM (et leurs IP/NIC) sans dupliquer le code. Changer `count` suffit à faire varier le nombre d'instances.
2. **Rôle de `custom_data` ?** Passer un script **cloud-init** (encodé en base64) exécuté au premier démarrage de la VM. Il automatise le provisioning (installation et configuration de Nginx, page d'accueil) sans intervention manuelle : la VM est opérationnelle dès le boot.



3. **Pourquoi `Standard_B1s` (ici `B2ts_v2`) est acceptable en formation ?** Ce sont des gabarits **burstable** à très faible coût, suffisants pour héberger un Nginx de démonstration sans charge réelle. On privilégie le coût sur la performance pour un environnement de dev/formation. ( `B1s` indisponible sur la souscription Students → `B2ts_v2` retenu.)
4. **Pourquoi éviter des IP publiques directes sur les VM en production ?** Chaque IP publique est une porte d'entrée supplémentaire à sécuriser et à superviser, augmentant la surface d'attaque. En prod, on n'expose que le Load Balancer / l'Application Gateway, on supprime les IP publiques des VM et on administre via Bastion. (Ici elles ne servent qu'à tester chaque VM individuellement.)

---

## Atelier 7 — Load Balancer

---

`loadbalancer.tf` crée l'IP publique du LB, le LB `Standard`, le backend pool (associé aux 2 NIC), une probe HTTP `/:80` et une règle TCP `80→80`.

### Réponses à l'analyse (11.3) :

1. **Quel problème résout le Load Balancer ?** Le **point de défaillance unique** (SPOF) et l'absence de répartition de charge : il distribue le trafic entrant sur plusieurs VM, améliorant la disponibilité et la capacité à absorber la charge.
2. **Que se passe-t-il si une VM devient indisponible ?** La **probe** de santé détecte l'échec (la VM ne répond plus sur `/:80`) et le LB cesse de lui router du trafic. Les requêtes sont automatiquement dirigées vers la VM saine restante : continuité de service (en mode dégradé).
3. **Différence avec Azure Application Gateway ?** Le Load Balancer opère en **couche 4** (TCP/UDP). L'Application Gateway opère en **couche 7** (HTTP/HTTPS) : routage par URL/host, terminaison TLS, **WAF** intégré, cookies d'affinité. L'AppGw est préférable pour exposer une application web en production ; le LB suffit pour une répartition réseau simple.

---

## Atelier 8 — Storage Account

---

`storage.tf` crée un `random_string` (suffixe d'unicité globale), le Storage Account (`Standard` / `LRS` / `TLS1_2` / versioning Blob activé) et un container **privé** `documents`.

### Réponses FinOps et sécurité (12.3) :

1. **Pourquoi le container doit-il être privé ?** Il contient des documents métier (factures, données clients). Un accès public anonyme exposerait des données potentiellement sensibles et constituerait une fuite (et une non-conformité RGPD). L'accès se fait via clés/SAS/identités gérées uniquement.
2. **Pourquoi le versioning peut-il augmenter les coûts ?** Chaque modification/suppression d'un blob conserve les **versions antérieures**, qui sont facturées comme

du stockage supplémentaire. Sur des objets fréquemment modifiés, le volume cumulé (et donc la facture) croît continuellement.

3. **Quelle règle de cycle de vie proposer ?** Une **lifecycle management policy** : transition des anciennes versions vers un tier froid (Cool/Archive) après N jours, puis **suppression** des versions au-delà d'une rétention définie (ex. supprimer les versions > 90 jours). Voir le détail dans `04_autonomie_subnet_prive.md` (section variante lifecycle).

## Atelier 9 — Outputs et validation

```
outputs.tf expose resource_group_name , load_balancer_public_ip , web_vm_public_ips ,
storage_account_name .
```

### Checklist technique (13.1)

**État** : déploiement réalisé sur Azure for Students le 25/06/2026. Tous les contrôles sont **OK**, chacun appuyé par une capture en Annexe C du PDF (dépôt dans `tp2/screenshots/`, voir `06_captures_a_faire.md`).

| Contrôle                                     | Statut | Vérification                                                                           | Capture                                                                   |
|----------------------------------------------|--------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Le projet Terraform s'initialise sans erreur | OK     | <code>terraform init</code> → Terraform has been successfully initialized!             | <code>atelier_02-init.png</code>                                          |
| <code>terraform validate</code> réussit      | OK     | Success! The configuration is valid.                                                   | <code>atelier_02-validate.png</code>                                      |
| Le Resource Group est créé                   | OK     | <code>rg-shopeasy-dev</code> dans Sweden Central (portail)                             | <code>atelier_04-portail-rg.png</code>                                    |
| Le VNet et les subnets existent              | OK     | <code>snet-web 10.20.1.0/24</code><br>+ <code>snet-data 10.20.2.0/24</code>            | <code>atelier_05-vnet-subnets.png</code>                                  |
| Le NSG limite SSH à votre IP                 | OK     | Règle <code>Allow-SSH-Admin</code><br>source = <code>216.252.179.39/32</code>          | <code>atelier_05-nsg-web.png</code>                                       |
| Deux VM Linux sont créées                    | OK     | <code>vm-shopeasy-dev-web-1/2</code> en Exécution (Standard_B2ts_v2)                   | <code>atelier_06-vms.png</code>                                           |
| Nginx répond sur les VM                      | OK     | <code>http://4.223.225.195</code> (web 1) et <code>http://4.223.111.127</code> (web 2) | <code>atelier_06-page-vm1.png</code> / <code>-vm2.png</code>              |
| Le Load Balancer répond en HTTP              | OK     | <code>http://20.91.219.236</code> → page ShopEasy (alternance web 1/2)                 | <code>atelier_09-page-lb.png</code> / <code>atelier_09-curl-lb.png</code> |

| Contrôle                      | Statut | Vérification                                                                           | Capture                             |
|-------------------------------|--------|----------------------------------------------------------------------------------------|-------------------------------------|
| Le Storage Account est privé  | OK     | Container <code>documents</code> niveau d'accès = <code>Privé</code>                   | <code>atelier_08-storage.png</code> |
| Le versioning Blob est activé | OK     | <code>blob_properties.versioning_enabled = true</code> (code <code>storage.tf</code> ) | <code>atelier_08-storage.png</code> |
| Les ressources sont taguées   | OK     | 5 tags <code>common_tags</code> visibles sur le RG                                     | <code>atelier_04-tags.png</code>    |

Toutes les captures sont intégrées automatiquement dans l'annexe « Preuves d'exécution » du PDF de rendu.

## Atelier 10 — Modifier l'infrastructure

Le tag `cost_center = "cloud-training"` est **déjà présent** dans `common_tags` (`locals.tf`). Pour l'exercice, on illustre l'ajout d'un tag supplémentaire (ex. `reviewed_by`).

### Réponses à l'analyse du plan (14.2) :

- 1. Terraform recrée-t-il toutes les ressources ?** Non. Un changement de tag est une modification **in-place** : Terraform affiche `~` (update) et ne touche pas au cycle de vie des ressources.
- 2. Quelles ressources sont simplement mises à jour ?** Toutes les ressources taguables référençant `local.common_tags` (RG, VNet, NSG, IP, NIC, VM, LB, Storage) : seul leur bloc `tags` est mis à jour.
- 3. Pourquoi le plan est-il indispensable ?** Il permet de **vérifier l'impact avant application** : distinguer une simple mise à jour (`~`) d'une recréation destructrice (`-/+`), repérer une suppression imprévue, contrôler les coûts et la conformité. C'est le filet de sécurité du workflow déclaratif.

## Atelier 11 — Observer une dérive (drift)

Procédure réalisée : ajout d'un tag manuel hors Terraform (`az tag update ... --tags manual_change=true` sur le RG), puis `terraform plan`. Résultat capturé dans `atelier_11-drift-plan.png`.

**Observation réelle** : Terraform a détecté la dérive et proposé `Plan: 0 to add, 1 to change, 0 to destroy`, avec sur le Resource Group :

```
~ tags = {
  - "manual_change" = "true" -> null
}
```

## Réponses à l'analyse (15.2) :

1. **Terraform détecte-t-il une différence ?** Oui. Au `plan`, il rafraîchit le state réel, constate l'écart entre l'infrastructure (tag manuel ajouté) et le code (qui ne le contient pas) et signale un changement ( `~ update in-place` ).
2. **Quelle action propose-t-il ?** De **revenir à l'état déclaré** : il prévoit de **supprimer** le tag `manual_change` ajouté à la main ( `"manual_change" = "true" -> null` ), car le code est la source de vérité.
3. **Pourquoi les modifications manuelles sont-elles dangereuses ?** Elles créent une **dérive** entre code et réel : l'infrastructure devient imprévisible, non reproductible, et les changements ne sont ni tracés ni revus. Un `apply` ultérieur peut écraser silencieusement une correction d'urgence faite à la main, ou inversement masquer un changement non documenté.
4. **Quelle règle d'équipe proposer ? Interdire les modifications manuelles en dehors des urgences** : tout changement passe par une **pull request** modifiant le code Terraform, relue puis appliquée via pipeline. Le portail reste réservé à l'observation/diagnostic. On peut renforcer avec **Azure Policy** et des `plan` planifiés détectant le drift.

---

## Atelier 12 — Préparer un state distant

### Réponses aux questions (16.2) :

1. **Pourquoi le state distant facilite-t-il le travail en équipe ?** Le state est **partagé** et centralisé (un Storage Account Azure), avec **verrouillage** (lock) empêchant deux `apply` simultanés de corrompre le state. Chacun travaille sur la même source de vérité, sans s'échanger un fichier local.
2. **Pourquoi protéger l'accès au Storage Account du state ?** Parce que le state contient la cartographie et des **secrets** de l'infrastructure. Un accès non maîtrisé permettrait fuite d'informations ou manipulation. On le protège par **RBAC**, chiffrement au repos, restriction réseau et journalisation.
3. **Pourquoi séparer le state dev / recette / prod ?** Pour **isoler les environnements** : un incident, un `destroy` ou une corruption sur le state de dev ne doit jamais impacter la prod. Cela permet aussi des droits d'accès différenciés (moindre privilège par environnement) via des `key` distinctes ( `shopeasy/dev/...` , `shopeasy/prod/...` ).

---

## Atelier 14 — Nettoyage

```
terraform plan -destroy # verifier ce qui sera detruit
terraform destroy      # supprimer toutes les ressources
```

Vérifier ensuite dans le portail que `rg-shopeasy-dev` est vide ou supprimé : aucune ressource facturable ne doit subsister.

---

## Synthèse

---

L'ensemble des ateliers est couvert par le projet Terraform ( `tp2/terraform/` ). Les analyses FinOps et sécurité approfondies sont dans [03\\_analyse\\_finops\\_securite.md](#) , l'extension autonomie (option A) dans [04\\_autonomie\\_subnet\\_prive.md](#) , et la note technique de synthèse dans [05\\_note\\_technique.md](#) .

# TP2 Terraform — Analyse coût, sécurité et maintenabilité

Atelier 13 du sujet. Cas ShopEasy, environnement de dev sur `swedencentral`.

## 1. Analyse FinOps (17.1)

| Ressource                              | Coût relatif                         | Risque de surcoût                                                         | Optimisation proposée                                                                                                                      |
|----------------------------------------|--------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VM Linux (×2)</b>                   | Élevé (poste principal)              | VM surdimensionnées ou laissées allumées 24/7                             | Gabarit burstable ( <code>B2ts_v2</code> ), arrêt/désallocation hors usage, auto-shutdown, <code>terraform destroy</code> en fin de séance |
| <b>IP publiques (×3 : 2 VM + 1 LB)</b> | Faible à modéré                      | IP Standard statiques facturées même inutilisées ; multiplication inutile | Supprimer les IP publiques des VM (accès via LB + Bastion) → ne garder que l'IP du LB                                                      |
| <b>Load Balancer</b>                   | Modéré (SKU Standard + règles)       | Règles/IP facturées en continu                                            | Mutualiser un seul LB pour plusieurs services ; détruire en dev hors usage                                                                 |
| <b>Storage Account</b>                 | Faible (volume de démo)              | Croissance non maîtrisée des données                                      | Tier Standard / LRS en dev, lifecycle policy, surveiller la volumétrie                                                                     |
| <b>Versioning Blob</b>                 | Faible mais cumulatif                | Accumulation des versions antérieures facturées                           | Lifecycle : transition Cool/ Archive puis suppression des versions > N jours                                                               |
| <b>Disques managés (OS)</b>            | Faible ( <code>Standard_LRS</code> ) | Disques orphelins après suppression de VM                                 | <code>Standard_LRS</code> en dev, nettoyage des disques non rattachés, <code>destroy</code> complet                                        |

**Leviers FinOps appliqués dans le projet :** taille de VM adaptée (variable `vm_size` ), tags de coût ( `cost_center` ), réplication `LRS` (la moins chère), et destruction reproductible via `terraform destroy`.

## 2. Analyse de sécurité (17.2)

| Risque                 | Cause possible | Impact                            | Correction                                               |
|------------------------|----------------|-----------------------------------|----------------------------------------------------------|
| <b>SSH trop ouvert</b> |                | Brute-force, intrusion sur les VM | SSH restreint à <code>allowed_ssh_cidr (/32)</code> ; en |

| Risque                       | Cause possible                                                                      | Impact                                       | Correction                                                                                                                                  |
|------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
|                              | Règle NSG en <code>0.0.0.0/0</code> sur le port 22                                  |                                              | prod, Azure Bastion / suppression du SSH public                                                                                             |
| <b>State exposé</b>          | <code>terraform.tfstate</code> commité ou partagé sans contrôle                     | Fuite de secrets et de la topologie          | <code>.gitignore</code> du state en local ; backend <code>azurerm</code> distant chiffré + RBAC en équipe                                   |
| <b>Storage public</b>        | <code>container_access_type</code> en <code>blob / container</code>                 | Fuite de documents métier (RGPD)             | Container <code>private</code> (appliqué) ; accès via SAS/identités gérées uniquement                                                       |
| <b>Tags absents</b>          | Ressources créées sans tags                                                         | Coûts non imputables, gouvernance impossible | <code>common_tags</code> appliqué à toutes les ressources taguables ; Azure Policy d'enforcement                                            |
| <b>Secrets dans Git</b>      | Mot de passe / clé en clair dans <code>.tf</code> ou <code>tf-vars</code> versionné | Compromission d'identifiants                 | Aucun secret en dur ; seule la clé SSH <b>publique</b> est référencée par chemin ; <code>terraform.tfvars</code> ignoré ; Key Vault en prod |
| <b>Modification manuelle</b> | Changement dans le portail hors Terraform                                           | Dérive (drift), perte de reproductibilité    | Changements par PR + plan / apply contrôlés ; plan périodique de détection ; Azure Policy                                                   |

### 3. Maintenabilité (17.3)

- Rendre le projet réutilisable pour la recette ?** Les valeurs sont déjà paramétrées par variables. Il suffit d'un jeu de valeurs par environnement : soit un fichier `recette.tfvars` (`environment = "recette"`, `vm_size`, `CIDR...`) appliqué via `terraform apply -var-file=recette.tfvars`, soit des **workspaces** Terraform, soit (mieux) un **backend** avec une `key` distincte par environnement. Le `prefix` (`shopeasy-<env>`) garantit des noms de ressources uniques par environnement.
- Quelles variables ajouter ?** `vm_count` (nombre de VM web), `os_disk_type`, `account_replication_type` (LRS/GRS selon l'env), `tags` additionnels (ex. `expiration`), un booléen `enable_public_ip_on_vm` pour couper les IP publiques en prod, et éventuellement `address_space` /préfixes déjà variabilisés.
- Quels fichiers pourraient devenir des modules ?** - `network.tf` + `security.tf` → **module network** (RG, VNet, subnets, NSG). - `compute.tf` → **module compute** (VM, NIC, IP, cloud-init). - `loadbalancer.tf` → **module loadbalancer**. - `storage.tf` → **module storage**. Le projet racine se réduirait alors à des appels de modules paramétrés par environnement.

4. **Quelles validations automatiques en CI/CD ?** `terraform fmt -check`, `terraform validate`, `terraform plan` posté en commentaire de PR, **analyse de sécurité statique** ( `tflint`, `tfsec / checkov` ), **estimation de coût** ( `infracost` ), scan de secrets (gitleaks), puis `apply` gated par approbation manuelle sur les environnements sensibles.



# TP2 Terraform — Mise en autonomie : Option A — Subnet privé pour les données

Atelier 19, option A du sujet. Extension de l'infrastructure ShopEasy : ajout d'un subnet privé destiné à accueillir une future base de données, sans aucune exposition publique.

## 1. Modification choisie

Ajouter au VNet un **subnet privé** `snet-data` ( `10.20.2.0/24` ) cloisonné par un **NSG dédié** `nsg-shopeasy-dev-data` :

- autorise **uniquement** le port **1433** (SQL Server) **depuis le subnet web** ( `10.20.1.0/24` ) ;
- **refuse tout autre trafic entrant** (règle `Deny-All-Inbound` priorité 4000) ;
- n'attache **aucune IP publique** : le subnet reste injoignable depuis Internet.

Ce choix prolonge naturellement l'architecture TP1 (qui prévoyait déjà une couche data isolée) et illustre la **segmentation réseau** (défense en profondeur) sans coût supplémentaire ni service complexe à déployer.

## 2. Fichiers impactés

| Fichier                   | Changement                                                                                                                                                                  |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>network.tf</code>   | Ajout de la ressource <code>azurerm_subnet.data</code> ( <code>snet-data</code> , <code>10.20.2.0/24</code> )                                                               |
| <code>security.tf</code>  | Ajout de <code>azurerm_network_security_group.data</code> (Allow-SQL-From-Web + Deny-All-Inbound) et de <code>azurerm_subnet_network_security_group_association.data</code> |
| <code>variables.tf</code> | Ajout de la variable <code>data_subnet_prefix</code> (défaut <code>10.20.2.0/24</code> )                                                                                    |

Extrait clé ( `security.tf` ) :

```
resource "azurerm_network_security_group" "data" {
  name = "nsg-${local.prefix}-data"
  # ...
  security_rule {
    name = "Allow-SQL-From-Web"
```

```

    priority          = 100
    destination_port_range = "1433"
    source_address_prefix = var.web_subnet_prefix
    # ...
  }
  security_rule {
    name          = "Deny-All-Inbound"
    priority      = 4000
    access        = "Deny"
    # ...
  }
}

```

### 3. Risques associés et points de vigilance

| Risque                            | Description                                                                                                          | Atténuation                                                                       |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Faux sentiment de sécurité</b> | Le subnet est isolé, mais une base réellement déployée nécessiterait aussi un private endpoint et un chiffrement     | Compléter avec Azure SQL + private endpoint + TLS lors de l'ajout réel de la base |
| <b>Règle trop large</b>           | Autoriser tout le subnet web vers 1433 reste plus permissif qu'un private endpoint nominatif                         | Restreindre ultérieurement aux NIC précises ou via Application Security Groups    |
| <b>Chevauchement d'adressage</b>  | 10.20.2.0/24 doit rester dans 10.20.0.0/16 et ne pas chevaucher <code>snet-web</code>                                | Préfixes gérés par variables, espaces disjoints vérifiés                          |
| <b>Subnet vide facturé ?</b>      | Un subnet seul n'est pas facturé, mais les futures ressources le seront                                              | Documenter ; destroy en fin de séance                                             |
| <b>Évolution vers PaaS</b>        | Pour Azure SQL managé, prévoir <code>service delegation</code> / <code>private endpoint</code> plutôt qu'une VM data | Anticipé dans la roadmap (voir note technique)                                    |

### 4. Variante envisagée — Option D (Storage lifecycle)

À titre complémentaire, une règle de cycle de vie limiterait le surcoût du versioning Blob (atelier 8). Elle pourrait être ajoutée dans `storage.tf` :

```

resource "azurerm_storage_management_policy" "docs" {
  storage_account_id = azurerm_storage_account.docs.id
  rule {
    name      = "expire-old-versions"
    enabled = true
    filters {

```

```
    blob_types = ["blockBlob"]
  }
  actions {
    version {
      delete_after_days_since_creation = 90
    }
    base_blob {
      tier_to_cool_after_days_since_modification_greater_than = 30
    }
  }
}
}
```

Option A retenue comme extension principale ; la variante D est documentée comme évolution FinOps possible.

# TP2 Terraform — Quiz de validation (réponses)

Réponses aux 20 questions de la section 21 du sujet TP2\_Terraform\_Azure.md .

1. **Terraform est-il impératif ou déclaratif ? Justifier. Déclaratif.** On décrit l'état final souhaité de l'infrastructure ; Terraform calcule lui-même la suite d'opérations (créer/modifier/détruire) nécessaire pour l'atteindre. On ne liste pas les étapes une à une.
2. **Rôle d'un provider Terraform ?** C'est un plugin qui traduit les ressources HCL en appels d'API d'une plateforme. Ici `azurerm` pilote l'API Azure Resource Manager ; `random` génère des valeurs aléatoires.
3. **À quoi sert `terraform.tfstate` ?** Il mémorise la correspondance entre les ressources déclarées dans le code et les ressources réellement créées dans le cloud (IDs, attributs). C'est la source de vérité de Terraform pour calculer les `plan` .
4. **Pourquoi `terraform plan` avant `terraform apply` ?** Pour prévisualiser les changements (créations `+` , modifications `~` , destructions `-` , remplacements `-/+` ) et détecter toute action dangereuse avant d'impacter Azure.
5. **Quelle commande formate le code ?** `terraform fmt` .
6. **Quelle commande vérifie la syntaxe et la cohérence ?** `terraform validate` .
7. **Quel service Azure correspond au réseau privé logique ?** Le **Virtual Network (VNet)**.
8. **Quel composant filtre les flux réseau entrants/sortants ?** Le **Network Security Group (NSG)**.
9. **Pourquoi restreindre SSH à une seule IP ?** Pour réduire drastiquement la surface d'attaque et limiter le brute-force : seul le poste de l'administrateur peut tenter une connexion.
10. **Intérêt de `count` pour les VM ?** Déployer plusieurs instances identiques depuis un seul bloc (indexées par `count.index` ), sans duplication de code, et faire varier facilement le nombre d'instances.
11. **Pourquoi utiliser des variables ?** Pour éviter les valeurs codées en dur, paramétrer le projet et le réutiliser sur plusieurs environnements (dev/test/prod) sans modifier le code.
12. **Pourquoi utiliser des outputs ?** Pour exposer après déploiement les informations utiles (IP du Load Balancer, nom du RG, nom du Storage) aux utilisateurs, scripts ou modules consommateurs.
13. **Pourquoi taguer les ressources ?** Pour la gouvernance et le FinOps : identifier le projet, l'environnement, le propriétaire et le centre de coût, faciliter le reporting des coûts et le cycle de vie.

14. **Différence entre un changement Terraform et un changement manuel dans le portail ?** Le changement Terraform est tracé (Git), revu, reproductible et applique l'état déclaré. Le changement manuel n'est pas tracé, crée une **dérive** (drift) et n'est pas reproductible.
15. **Quel risque présente un Storage Account public ?** Fuite de données : accès anonyme aux blobs (documents métier, données clients), exfiltration et non-conformité (RGPD).
16. **Pourquoi le versioning Blob peut-il générer des coûts ?** Chaque version antérieure d'un blob est conservée et facturée comme du stockage supplémentaire ; le volume cumulé croît à chaque modification.
17. **À quoi sert un Load Balancer ?** À répartir le trafic entre plusieurs VM et améliorer la disponibilité (bascule automatique via sonde de santé en cas de défaillance d'une instance).
18. **Pourquoi un state distant est-il préférable en équipe ?** State partagé et centralisé, avec verrouillage (évite les `apply` concurrents corrompant le state), sécurisable (RBAC, chiffrement) et intégrable en CI/CD.
19. **Deux bonnes pratiques de sécurité pour un projet Terraform.** (a) Ne jamais stocker de secrets dans le code/ `tfvars` versionné (Key Vault, variables d'environnement, variables CI/CD sécurisées). (b) Restreindre les accès réseau (SSH limité par IP, pas de `0.0.0.0/0`, Storage privé) et protéger le state distant par RBAC.
20. **Deux améliorations pour se rapprocher d'un environnement de production.** (a) Supprimer les IP publiques des VM et administrer via **Azure Bastion** ; externaliser le state dans un **backend Azure** sécurisé. (b) Ajouter une **Application Gateway + WAF**, un **subnet privé** pour les données, une pipeline **CI/CD** ( `fmt` / `validate` / `plan` / approbation) et de l'**observabilité** (Azure Monitor / Log Analytics).

# Annexe A — Arborescence du projet Terraform

---

```
cloud/tp2/
├─ terraform/
│   ├─ versions.tf
│   ├─ providers.tf
│   ├─ variables.tf
│   ├─ locals.tf
│   ├─ network.tf          (RG + VNet + snet-web + snet-data)
│   ├─ security.tf        (NSG web + NSG data + associations)
│   ├─ compute.tf         (2 VM Linux + NIC + IP publiques)
│   ├─ loadbalancer.tf    (LB + backend pool + probe + regle)
│   ├─ storage.tf         (Storage Account prive + container + versioning)
│   ├─ outputs.tf
│   ├─ terraform.tfvars.example
│   ├─ .gitignore
│   └─ templates/
│       └─ cloud-init.yml
├─ livrables/
│   ├─ 01_compte_rendu_ateliers.md
│   ├─ 02_quiz_reponses.md
│   ├─ 03_analyse_finops_securite.md
│   ├─ 04_autonomie_subnet_prive.md
│   ├─ 05_note_technique.md
│   └─ 06_captures_a_faire.md
└─ screenshots/          (preuves d'execution – Annexe C)
```

## Annexe B — Code source Terraform

---

Code intégral du projet `tp2/terraform/`. Les secrets ne figurent jamais dans le code : seule la clé SSH publique est référencée par chemin, et `terraform.tfvars` est ignoré par Git.

### versions.tf

---

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
    random = {
      source  = "hashicorp/random"
      version = "~> 3.6"
    }
  }
}
```

### providers.tf

---

```
provider "azurerm" {
  features {}

  # Epingle explicitement la souscription cible : Terraform agit sur CET abonnement,
  # indépendamment du compte par défaut de `az` (garde-fou anti-déploiement sur le mauvais compte).
  subscription_id = var.subscription_id
}
```

### variables.tf

---

```
variable "subscription_id" {
  description = "ID de la souscription Azure cible (compte de FORMATION). Sans valeur par défaut : il DOIT être renseigné dans terraform.tfvars pour éviter tout déploiement accidentel sur un abonnement d'entreprise."
  type        = string

  validation {
```

```

    condition      = can(regex("[0-9a-fA-F-]{36}$", var.subscription_id))
    error_message = "subscription_id doit etre un GUID Azure (36 caracteres). Recupere-le avec : az
account show --query id -o tsv (apres avoir selectionne le bon compte)."
  }
}

variable "project" {
  description = "Nom court du projet"
  type        = string
  default     = "shopeasy"
}

variable "environment" {
  description = "Nom de l'environnement (dev, test, prod)"
  type        = string
  default     = "dev"
}

variable "location" {
  description = "Region Azure cible. swedencentral est retenue car la policy Azure for Students
interdit francecentral."
  type        = string
  default     = "swedencentral"
}

variable "vm_size" {
  description = "Gabarit des VM web. Standard_B2ts_v2 est utilise car Standard_B1s est indisponible
sur la souscription Students."
  type        = string
  default     = "Standard_B2ts_v2"
}

variable "admin_username" {
  description = "Utilisateur administrateur Linux"
  type        = string
  default     = "azureuser"
}

variable "ssh_public_key_path" {
  description = "Chemin local vers la cle publique SSH"
  type        = string
}

variable "allowed_ssh_cidr" {
  description = "CIDR autorise pour SSH (idealement l'IP publique de l'apprenant en /32)"
  type        = string
}

```



```

}

variable "vnet_address_space" {
  description = "Espace d'adressage du Virtual Network"
  type        = string
  default     = "10.20.0.0/16"
}

variable "web_subnet_prefix" {
  description = "Prefixe du subnet applicatif web"
  type        = string
  default     = "10.20.1.0/24"
}

variable "data_subnet_prefix" {
  description = "Prefixe du subnet prive de donnees (mise en autonomie - option A)"
  type        = string
  default     = "10.20.2.0/24"
}

```

## locals.tf

---

```

locals {
  prefix = "${var.project}-${var.environment}"

  common_tags = {
    project      = var.project
    environment   = var.environment
    owner        = "formation"
    managed_by   = "terraform"
    cost_center  = "cloud-training"
  }
}

```

## network.tf

---

```

resource "azurerm_resource_group" "main" {
  name     = "rg-${local.prefix}"
  location = var.location
  tags     = local.common_tags
}

resource "azurerm_virtual_network" "main" {

```

```

name                = "vnet-`${local.prefix}`"
address_space       = [var.vnet_address_space]
location            = azurerm_resource_group.main.location
resource_group_name = azurerm_resource_group.main.name
tags                = local.common_tags
}

resource "azurerm_subnet" "web" {
  name                = "snet-web"
  resource_group_name = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes    = [var.web_subnet_prefix]
}

# Mise en autonomie - Option A : subnet prive destine a une future base de donnees.
# Il n'expose aucune ressource publique et est filtre par nsg-data (voir security.tf).
resource "azurerm_subnet" "data" {
  name                = "snet-data"
  resource_group_name = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes    = [var.data_subnet_prefix]
}

```

## security.tf

```

resource "azurerm_network_security_group" "web" {
  name                = "nsg-`${local.prefix}`-web"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags                = local.common_tags

  security_rule {
    name                = "Allow-HTTP"
    priority            = 100
    direction           = "Inbound"
    access              = "Allow"
    protocol            = "Tcp"
    source_port_range   = "*"
    destination_port_range = "80"
    source_address_prefix = "Internet"
    destination_address_prefix = "*"
  }

  security_rule {

```

```

    name                = "Allow-SSH-Admin"
    priority             = 110
    direction           = "Inbound"
    access              = "Allow"
    protocol             = "Tcp"
    source_port_range    = "*"
    destination_port_range = "22"
    source_address_prefix = var.allowed_ssh_cidr
    destination_address_prefix = "*"
  }
}

resource "azurerm_subnet_network_security_group_association" "web" {
  subnet_id            = azurerm_subnet.web.id
  network_security_group_id = azurerm_network_security_group.web.id
}

# Mise en autonomie - Option A : NSG du subnet prive de donnees.
# Seul le port 1433 (SQL) est autorise et uniquement depuis le subnet web.
# Aucun acces entrant depuis Internet : le subnet data reste prive.
resource "azurerm_network_security_group" "data" {
  name                = "nsg-${local.prefix}-data"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags                = local.common_tags

  security_rule {
    name                = "Allow-SQL-From-Web"
    priority            = 100
    direction           = "Inbound"
    access              = "Allow"
    protocol            = "Tcp"
    source_port_range    = "*"
    destination_port_range = "1433"
    source_address_prefix = var.web_subnet_prefix
    destination_address_prefix = "*"
  }

  security_rule {
    name                = "Deny-All-Inbound"
    priority            = 4000
    direction           = "Inbound"
    access              = "Deny"
    protocol            = "*"
    source_port_range    = "*"
    destination_port_range = "*"
  }
}

```

```

    source_address_prefix      = ""
    destination_address_prefix = ""
  }
}

resource "azurerm_subnet_network_security_group_association" "data" {
  subnet_id            = azurerm_subnet.data.id
  network_security_group_id = azurerm_network_security_group.data.id
}

```

## compute.tf

```

resource "azurerm_public_ip" "web" {
  count                = 2
  name                 = "pip-${local.prefix}-web-${count.index + 1}"
  location             = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method    = "Static"
  sku                  = "Standard"
  tags                 = local.common_tags
}

resource "azurerm_network_interface" "web" {
  count                = 2
  name                 = "nic-${local.prefix}-web-${count.index + 1}"
  location             = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags                 = local.common_tags

  ip_configuration {
    name                        = "internal"
    subnet_id                  = azurerm_subnet.web.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id        = azurerm_public_ip.web[count.index].id
  }
}

resource "azurerm_linux_virtual_machine" "web" {
  count                = 2
  name                 = "vm-${local.prefix}-web-${count.index + 1}"
  location             = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  size                 = var.vm_size
  admin_username       = var.admin_username
}

```

```

network_interface_ids = [azurerm_network_interface.web[count.index].id]

admin_ssh_key {
  username    = var.admin_username
  public_key  = file(var.ssh_public_key_path)
}

os_disk {
  caching              = "ReadWrite"
  storage_account_type = "Standard_LRS"
}

source_image_reference {
  publisher = "Canonical"
  offer     = "0001-com-ubuntu-server-jammy"
  sku       = "22_04-lts"
  version   = "latest"
}

custom_data = base64encode(templatefile("${path.module}/templates/cloud-init.yml", {
  server_index = count.index + 1
}))

tags = local.common_tags
}

```

## loadbalancer.tf

```

resource "azurerm_public_ip" "lb" {
  name                = "pip-${local.prefix}-lb"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method   = "Static"
  sku                 = "Standard"
  tags               = local.common_tags
}

resource "azurerm_lb" "web" {
  name                = "lb-${local.prefix}-web"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  sku                 = "Standard"
  tags               = local.common_tags
}

```

```

frontend_ip_configuration {
  name          = "public-frontend"
  public_ip_address_id = azurerm_public_ip.lb.id
}
}

resource "azurerm_lb_backend_address_pool" "web" {
  name          = "backend-web"
  loadbalancer_id = azurerm_lb.web.id
}

resource "azurerm_network_interface_backend_address_pool_association" "web" {
  count          = 2
  network_interface_id = azurerm_network_interface.web[count.index].id
  ip_configuration_name = "internal"
  backend_address_pool_id = azurerm_lb_backend_address_pool.web.id
}

resource "azurerm_lb_probe" "http" {
  name          = "http-probe"
  loadbalancer_id = azurerm_lb.web.id
  protocol      = "Http"
  request_path  = "/"
  port          = 80
}

resource "azurerm_lb_rule" "http" {
  name          = "http-rule"
  loadbalancer_id = azurerm_lb.web.id
  protocol      = "Tcp"
  frontend_port  = 80
  backend_port   = 80
  frontend_ip_configuration_name = "public-frontend"
  backend_address_pool_ids       = [azurerm_lb_backend_address_pool.web.id]
  probe_id                      = azurerm_lb_probe.http.id
}

```

## storage.tf

```

# Les noms de Storage Account doivent etre globalement uniques (3-24 caracteres, minuscules/
chiffres).
resource "random_string" "suffix" {
  length = 6
  upper  = false
}

```

```

    special = false
}

resource "azurerm_storage_account" "docs" {
  name                        = "${replace(local.prefix, "-", "")}docs${random_string.suffix.result}"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  account_tier               = "Standard"
  account_replication_type   = "LRS"
  min_tls_version            = "TLS1_2"
  tags                      = local.common_tags

  blob_properties {
    versioning_enabled = true
  }
}

resource "azurerm_storage_container" "documents" {
  name                = "documents"
  storage_account_id = azurerm_storage_account.docs.id
  container_access_type = "private"
}

```

## outputs.tf

```

output "resource_group_name" {
  description = "Nom du Resource Group cree"
  value       = azurerm_resource_group.main.name
}

output "load_balancer_public_ip" {
  description = "Adresse IP publique du Load Balancer (point d'entree de l'application)"
  value       = azurerm_public_ip.lb.ip_address
}

output "web_vm_public_ips" {
  description = "Adresses IP publiques directes des deux VM web"
  value       = azurerm_public_ip.web[*].ip_address
}

output "storage_account_name" {
  description = "Nom du Storage Account des documents"
  value       = azurerm_storage_account.docs.name
}

```

## terraform.tfvars.example

```
# Copier ce fichier en terraform.tfvars puis adapter les valeurs.
# terraform.tfvars est ignore par Git (peut contenir des informations propres a l'environnement).

# OBLIGATOIRE : ID de la souscription de FORMATION (jamais l'abonnement d'entreprise).
# Recuperer la bonne valeur APRES avoir selectionne le compte student :
#   ./scripts/azure-account.sh use-lab          (ou: az account set --subscription "<nom student>")
#   az account show --query id -o tsv
subscription_id = "00000000-0000-0000-0000-000000000000"

project          = "shopeasy"
environment      = "dev"
location         = "swedencentral"
vm_size          = "Standard_B2ts_v2"
admin_username   = "azureuser"
ssh_public_key_path = "~/ssh/id_rsa.pub"

# Remplacer par votre IP publique en /32 (ex: curl ifconfig.me) - ne jamais mettre 0.0.0.0/0.
allowed_ssh_cidr = "X.X.X.X/32"
```

## templates/cloud-init.yml

```
#cloud-config
package_update: true
packages:
  - nginx
write_files:
  - path: /var/www/html/index.html
    content: |
      <html>
      <body>
      <h1>ShopEasy - serveur web ${server_index}</h1>
      <p>Deploiement realise avec Terraform sur Azure.</p>
      </body>
      </html>
runcmd:
  - systemctl enable nginx
  - systemctl restart nginx
```

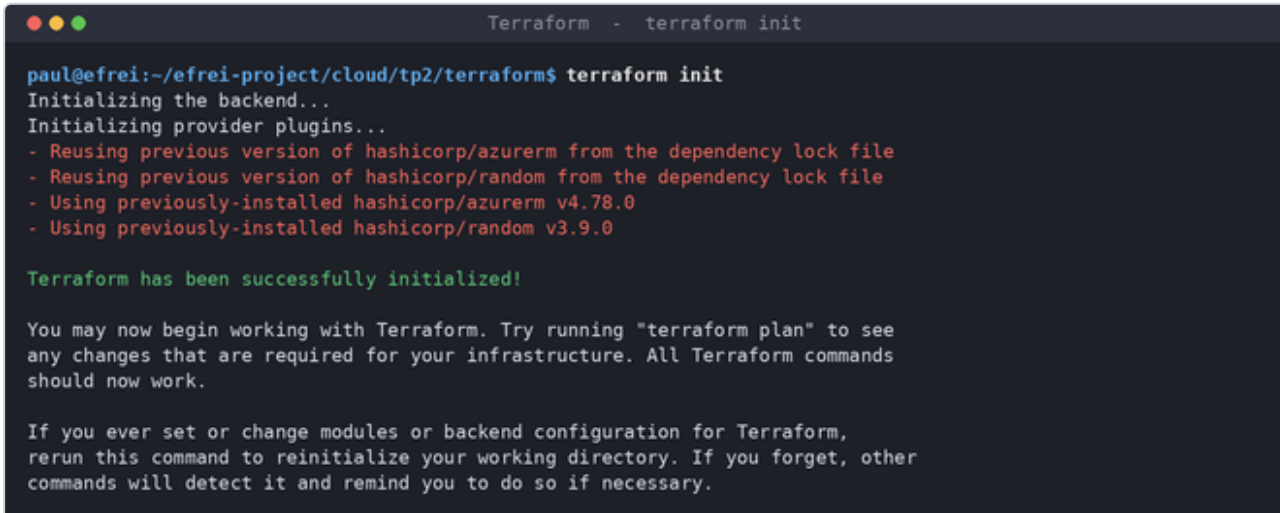


# Annexe C — Preuves d'exécution

---

Captures du workflow Terraform, du portail Azure et de la page web servie par le Load Balancer. La correspondance capture → atelier est détaillée dans `tp2/livrables/06_captures_a_faire.md`.

## atelier\_02-init.png



```
Terraform - terraform init

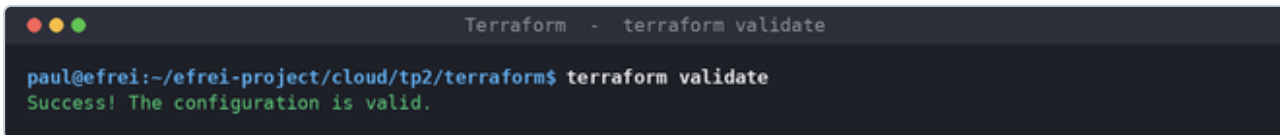
paul@efrei:~/efrei-project/cloud/tp2/terraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Reusing previous version of hashicorp/random from the dependency lock file
- Using previously-installed hashicorp/azurerm v4.78.0
- Using previously-installed hashicorp/random v3.9.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

## atelier\_02-validate.png



```
Terraform - terraform validate

paul@efrei:~/efrei-project/cloud/tp2/terraform$ terraform validate
Success! The configuration is valid.
```

```

Terraform - terraform plan

paul@efrei:~/efrei-project/cloud/tp2/terraform$ terraform plan
Terraform will perform the following actions:

# azurerm_lb.web will be created
# azurerm_lb_backend_address_pool.web will be created
# azurerm_lb_probe.http will be created
# azurerm_lb_rule.http will be created
# azurerm_linux_virtual_machine.web[0] will be created
# azurerm_linux_virtual_machine.web[1] will be created
# azurerm_network_interface.web[0] will be created
# azurerm_network_interface.web[1] will be created
# azurerm_network_interface_backend_address_pool_association.web[0] will be created
# azurerm_network_interface_backend_address_pool_association.web[1] will be created
# azurerm_network_security_group.data will be created
# azurerm_network_security_group.web will be created
# azurerm_public_ip.lb will be created
# azurerm_public_ip.web[0] will be created
# azurerm_public_ip.web[1] will be created
# azurerm_resource_group.main will be created
# azurerm_storage_account.docs will be created
# azurerm_storage_container.documents will be created
# azurerm_subnet.data will be created
# azurerm_subnet.web will be created
# azurerm_subnet_network_security_group_association.data will be created
# azurerm_subnet_network_security_group_association.web will be created
# azurerm_virtual_network.main will be created
# random_string.suffix will be created

Plan: 24 to add, 0 to change, 0 to destroy.

```

## atelier\_04-apply.png

```
Terraform - terraform apply

paul@efrei:~/efrei-project/cloud/tp2/terraform$ terraform apply
random_string.suffix: Creation complete after 0s
azurerm_resource_group.main: Creation complete after 26s
azurerm_public_ip.web[0]: Creation complete after 7s
azurerm_network_security_group.data: Creation complete after 8s
azurerm_public_ip.lb: Creation complete after 8s
azurerm_public_ip.web[1]: Creation complete after 8s
azurerm_network_security_group.web: Creation complete after 8s
azurerm_virtual_network.main: Creation complete after 9s
azurerm_subnet.web: Creation complete after 6s
azurerm_lb.web: Creation complete after 13s
azurerm_subnet.data: Creation complete after 11s
azurerm_subnet_network_security_group_association.web: Creation complete after 12s
azurerm_network_interface.web[0]: Creation complete after 14s
azurerm_lb_probe.http: Creation complete after 12s
azurerm_network_interface.web[1]: Creation complete after 17s
azurerm_lb_backend_address_pool.web: Creation complete after 14s
azurerm_network_interface_backend_address_pool_association.web[1]: Creation complete after 3s
azurerm_network_interface_backend_address_pool_association.web[0]: Creation complete after 3s
azurerm_subnet_network_security_group_association.data: Creation complete after 17s
azurerm_lb_rule.http: Creation complete after 6s
azurerm_storage_account.docs: Creation complete after 1m7s
azurerm_storage_container.documents: Creation complete after 11s
azurerm_linux_virtual_machine.web[0]: Creation complete after 49s
azurerm_linux_virtual_machine.web[1]: Creation complete after 49s
Apply complete! Resources: 24 added, 0 changed, 0 destroyed.

Outputs:

load_balancer_public_ip = "20.91.219.236"
resource_group_name = "rg-shopeasy-dev"
storage_account_name = "shopeasydevdocsbvhvip"
web_vm_public_ips = [
  "4.223.225.195",
  "4.223.111.127",
]
```

## atelier\_04-portail-rg.png

Microsoft Azure

Rechercher dans les ressources, services et documents (G+)

Copilot

rg-shopeasy-dev

Générez du code Bicep pour dupliquer ce groupe de ressources. Comment gérer ces modifications plus efficacement avec les outils de déploiement ?

Rechercher

+ Créer Gérer l'affichage Supprimer le groupe de ressources Actualiser Exporter au format CSV

Aucun regroupement

Vue d'ensemble

Journal d'activité

Contrôle d'accès (IAM)

Étiquettes

Visualiseur de ressources

Événements

Paramètres

Gestion des coûts

Supervision

Automatisation

Aide

Bases

Ressources

Recommandations

Filtrer un champ...

Type est égal à tout

Emplacement est égal à tout

Ajouter un filtre

| Nom                    | Type                  | Emplacement    |
|------------------------|-----------------------|----------------|
| lb-shopeasy-dev-web    | Équilibreur de charge | Sweden Central |
| nic-shopeasy-dev-web-1 | Interface réseau      | Sweden Central |
| nic-shopeasy-dev-web-2 | Interface réseau      | Sweden Central |
| nsg-shopeasy-dev-data  | Groupe de sécurité    | Sweden Central |
| nsg-shopeasy-dev-web   | Groupe de sécurité    | Sweden Central |
| pip-shopeasy-dev-lb    | Adresse IP publique   | Sweden Central |
| pip-shopeasy-dev-web-1 | Adresse IP publique   | Sweden Central |
| pip-shopeasy-dev-web-2 | Adresse IP publique   | Sweden Central |
| shopeasydevdocsbhvip   | Compte de stockage    | Sweden Central |
| vm-shopeasy-dev-web-1  | Machine virtuelle     | Sweden Central |

Affichage de 1 - 10 sur 14. Afficher le nombre : auto

Envoyer des commentaires

## atelier\_04-tags.png

Microsoft Azure

Rechercher dans les ressources, services et documents (G+)

Copilot

rg-shopeasy-dev | Étiquettes

Tout supprimer Commentaires

Les étiquettes sont des paires nom/valeur qui vous permettent de catégoriser les ressources et de voir une facturation centralisée en appliquant la même étiquette à plusieurs ressources et groupes de ressources. Les noms d'étiquette ne sont pas sensibles à la casse alors que les valeurs d'étiquette le sont. En savoir plus sur les étiquettes.

N'entrez pas de noms ou de valeurs qui pourraient affaiblir la sécurité de vos ressources ou contenir des informations personnelles/sensibles, car les données d'étiquette sont répliquées à l'échelle mondiale.

| Nom         | Valeur         |
|-------------|----------------|
| cost_center | cloud-training |
| environment | dev            |
| managed_by  | terraform      |
| owner       | formation      |
| project     | shopeasy       |

rg-shopeasy-dev (Groupe de ressources)

cost\_center : cloud-training environment : dev managed\_by : terraform owner : formation project : shopeasy

Aucune modification

Appliquer Ignorer les changements

## atelier\_05-nsg-data.png

The screenshot shows the Azure portal interface for a Network Security Group (NSG) named 'nsg-shopeasy-dev-data'. The left sidebar contains navigation options like 'Vue d'ensemble', 'Journal d'activité', 'Contrôle d'accès (IAM)', 'Étiquettes', 'Diagnostic et résoudre les problèmes', 'Visualiseur de ressources', 'Paramètres', 'Supervision', 'Automatisation', and 'Aide'. The main content area displays the NSG configuration details, including its location (Sweden Central), subscription (Azure for Students), and associated virtual network. Below this, there are tabs for 'Règles de sécurité de trafic entrant' and 'Règles de sécurité de trafic sortant'. The 'Règles de sécurité de trafic entrant' tab is active, showing a table of inbound rules.

| Priorité | Nom                     | Port             | Protocole        | Source            | Destination      | Action |
|----------|-------------------------|------------------|------------------|-------------------|------------------|--------|
| 100      | Allow-SQL-From-Web      | 1433             | Tcp              | 10.20.1.0/24      | N'importe lequel | Allow  |
| 4000     | Deny-All-Inbound        | N'importe lequel | N'importe lequel | N'importe lequel  | N'importe lequel | Deny   |
| 65000    | AllowVnetInBound        | N'importe lequel | N'importe lequel | VirtualNetwork    | VirtualNetwork   | Allow  |
| 65001    | AllowAzureLoadBalanc... | N'importe lequel | N'importe lequel | AzureLoadBalancer | N'importe lequel | Allow  |
| 65500    | DenyAllInBound          | N'importe lequel | N'importe lequel | N'importe lequel  | N'importe lequel | Deny   |

The 'Règles de sécurité de trafic sortant' tab is also visible, showing a table of outbound rules.

| Priorité | Nom                   | Port             | Protocole        | Source           | Destination      | Action |
|----------|-----------------------|------------------|------------------|------------------|------------------|--------|
| 65000    | AllowVnetOutBound     | N'importe lequel | N'importe lequel | VirtualNetwork   | VirtualNetwork   | Allow  |
| 65001    | AllowInternetOutBound | N'importe lequel | N'importe lequel | N'importe lequel | Internet         | Allow  |
| 65500    | DenyAllOutBound       | N'importe lequel | N'importe lequel | N'importe lequel | N'importe lequel | Deny   |

## atelier\_05-nsg-web.png

The screenshot shows the Azure portal interface for a Network Security Group (NSG) named 'nsg-shopeasy-dev-web'. The left sidebar contains navigation options like 'Vue d'ensemble', 'Journal d'activité', 'Contrôle d'accès (IAM)', 'Étiquettes', 'Diagnostic et résoudre les problèmes', 'Visualiseur de ressources', 'Paramètres', 'Supervision', 'Automatisation', and 'Aide'. The main content area displays the NSG configuration details, including its location (Sweden Central), subscription (Azure for Students), and associated virtual network. Below this, there are tabs for 'Règles de sécurité de trafic entrant' and 'Règles de sécurité de trafic sortant'. The 'Règles de sécurité de trafic entrant' tab is active, showing a table of inbound rules.

| Priorité | Nom                     | Port             | Protocole        | Source            | Destination      | Action |
|----------|-------------------------|------------------|------------------|-------------------|------------------|--------|
| 100      | Allow-HTTP              | 80               | Tcp              | Internet          | N'importe lequel | Allow  |
| 110      | Allow-SSH-Admin         | 22               | Tcp              | 216.252.179.39/32 | N'importe lequel | Allow  |
| 65000    | AllowVnetInBound        | N'importe lequel | N'importe lequel | VirtualNetwork    | VirtualNetwork   | Allow  |
| 65001    | AllowAzureLoadBalanc... | N'importe lequel | N'importe lequel | AzureLoadBalancer | N'importe lequel | Allow  |
| 65500    | DenyAllInBound          | N'importe lequel | N'importe lequel | N'importe lequel  | N'importe lequel | Deny   |

The 'Règles de sécurité de trafic sortant' tab is also visible, showing a table of outbound rules.

| Priorité | Nom                   | Port             | Protocole        | Source           | Destination      | Action |
|----------|-----------------------|------------------|------------------|------------------|------------------|--------|
| 65000    | AllowVnetOutBound     | N'importe lequel | N'importe lequel | VirtualNetwork   | VirtualNetwork   | Allow  |
| 65001    | AllowInternetOutBound | N'importe lequel | N'importe lequel | N'importe lequel | Internet         | Allow  |
| 65500    | DenyAllOutBound       | N'importe lequel | N'importe lequel | N'importe lequel | N'importe lequel | Deny   |

## atelier\_05-vnet-subnets.png

The screenshot shows the Microsoft Azure portal interface. The top navigation bar includes the 'Microsoft Azure' logo, a search bar, and the user profile 'paul.claverie@efrei.net'. The left sidebar contains a list of navigation items, with 'Sous-réseaux' (Subnets) selected. The main content area displays the 'vnet-shopeasy-dev | Sous-réseaux' page. It includes a search bar, a '+ Sous-réseau' button, and an 'Actualiser' button. Below this, a table lists the subnets:

| Nom       | IPv4         | IPv6 | Adresses IP dis... | Délégué à | Groupe de séc... | Table de routa... |
|-----------|--------------|------|--------------------|-----------|------------------|-------------------|
| snet-web  | 10.20.1.0/24 | -    | 249                | -         | nsg-shope...     | -                 |
| snet-data | 10.20.2.0/24 | -    | 251                | -         | nsg-shope...     | -                 |

At the bottom of the page, it indicates 'Affichage des sous-réseaux 2' and provides a link to 'Envoyer des commentaires'.

## atelier\_06-page-vm1.png

The screenshot shows a page titled 'ShopEasy - serveur web 1'. Below the title, it states 'Déploiement réalisé avec Terraform sur Azure.' The page is mostly blank, suggesting it might be a placeholder or a page with content that is not visible in the screenshot.

## atelier\_06-page-vm2.png

### ShopEasy - serveur web 2

Deploiement realise avec Terraform sur Azure.

## atelier\_06-state-list.png

```
Terraform - terraform state list

paul@efrei:~/efrei-project/cloud/tp2/terraform$ terraform state list
azurerm_lb.web
azurerm_lb_backend_address_pool.web
azurerm_lb_probe.http
azurerm_lb_rule.http
azurerm_linux_virtual_machine.web[0]
azurerm_linux_virtual_machine.web[1]
azurerm_network_interface.web[0]
azurerm_network_interface.web[1]
azurerm_network_interface_backend_address_pool_association.web[0]
azurerm_network_interface_backend_address_pool_association.web[1]
azurerm_network_security_group.data
azurerm_network_security_group.web
azurerm_public_ip.lb
azurerm_public_ip.web[0]
azurerm_public_ip.web[1]
azurerm_resource_group.main
azurerm_storage_account.docs
azurerm_storage_container.documents
azurerm_subnet.data
azurerm_subnet.web
azurerm_subnet_network_security_group_association.data
azurerm_subnet_network_security_group_association.web
azurerm_virtual_network.main
random_string.suffix
```

## atelier\_06-vms.png

Microsoft Azure

Rechercher dans les ressources, services et documents (G+)

Copilot

Accueil

Machines virtuelles

Afficher les machines virtuelles avec des alertes critiques

Analyser les alertes sur les machines virtuelles

+1

+ Créer

Réervations

Gérer l'affichage

Actualiser

Exporter au format CSV

Ouvrez une requête

Attribuer des balises

Démarrer

Aucun regroupement

Filtrer un champ...

Abonnement est égal à tout

Type est égal à tout

Groupe de ressources est égal à tout

Emplacement est égal à tout

+ Ajouter un filtre

|                          | Nom                   | Abonnement         | Groupe de res... | Emplacement    | État      | Système d'expl... | Taille           | Adresse IP pub... | Disques | Mettre à jour l'...  |
|--------------------------|-----------------------|--------------------|------------------|----------------|-----------|-------------------|------------------|-------------------|---------|----------------------|
| <input type="checkbox"/> | vm-shopeasy-dev-web-1 | Azure for Stude... | rg-shopeasy-dev  | Sweden Central | Exécution | Linux             | Standard_B2ts_v2 | 4.223.225.195     | 1       | Activer l'évaluat... |
| <input type="checkbox"/> | vm-shopeasy-dev-web-2 | Azure for Stude... | rg-shopeasy-dev  | Sweden Central | Exécution | Linux             | Standard_B2ts_v2 | 4.223.111.127     | 1       | Activer l'évaluat... |

Affichage de 1 – 2 sur 2. Afficher le nombre : auto

Envoyer des commentaires

## atelier\_07-lb-backend.png

Microsoft Azure

Rechercher dans les ressources, services et documents (G+)

Copilot

Accueil

lb-shopeasy-dev-web | Pools principaux

Équilibreur de charge

Rechercher

+ Ajouter

Actualiser

Exporter au format CSV

Le pool de back-end est un composant critique de l'équilibreur de charge. Il définit le groupe de ressources qui servira le trafic pour une règle d'équilibrage de charge donnée. [Découvrez plus d'informations.](#)

Ajouter un filtre

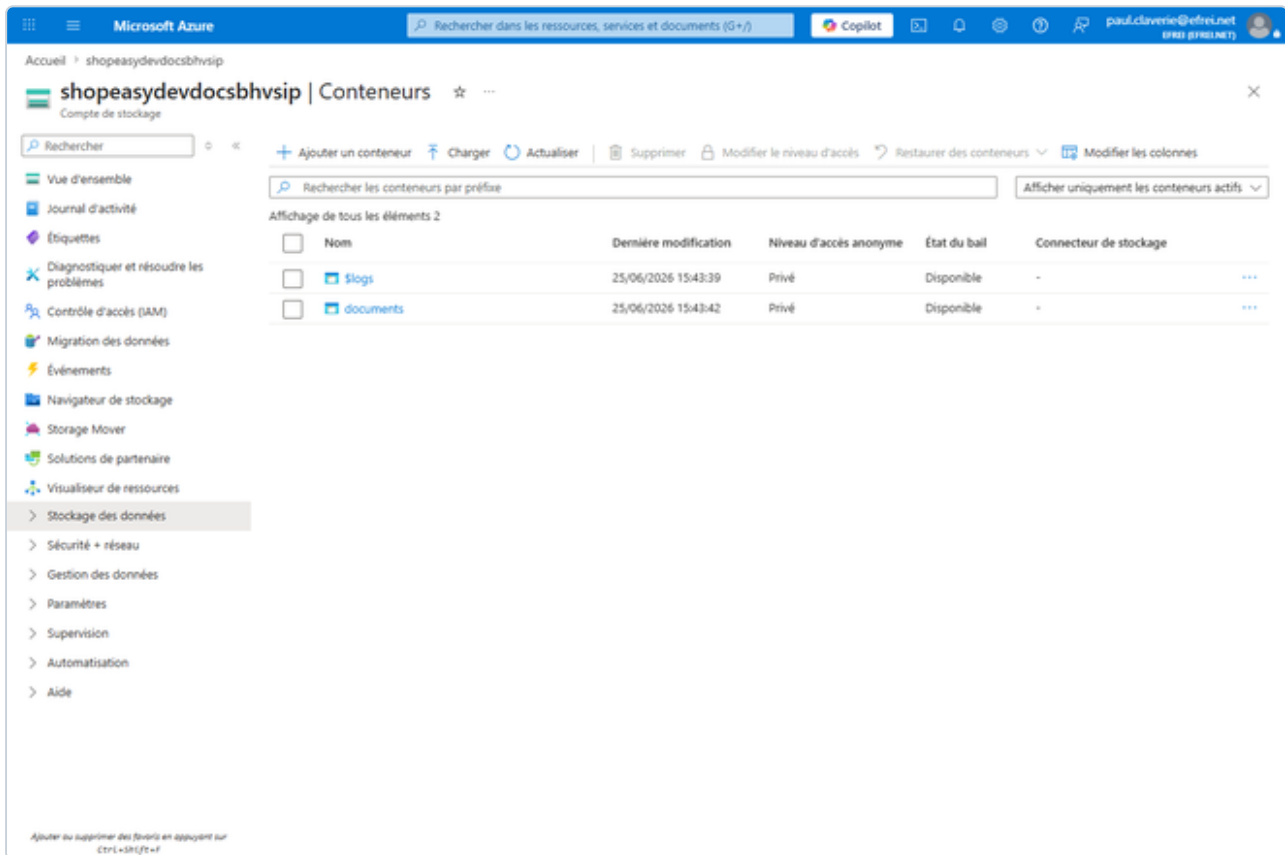
|   | Pool de back-ends | Nom de ressource   | Adresse IP | Interface réseau    | Zone de disponib... | Nombre de règles | État de la ressource | État d'administ |
|---|-------------------|--------------------|------------|---------------------|---------------------|------------------|----------------------|-----------------|
| ▼ | backend-web (2)   |                    |            |                     |                     |                  |                      |                 |
|   | backend-web       | vm-shopeasy-dev-we | 10.20.1.5  | nic-shopeasy-dev-we | -                   | 1                | Exécution            | Aucun           |
|   | backend-web       | vm-shopeasy-dev-we | 10.20.1.4  | nic-shopeasy-dev-we | -                   | 1                | Exécution            | Aucun           |

Ajouter au supprimer des favoris en appuyant sur Ctrl+Shift+F

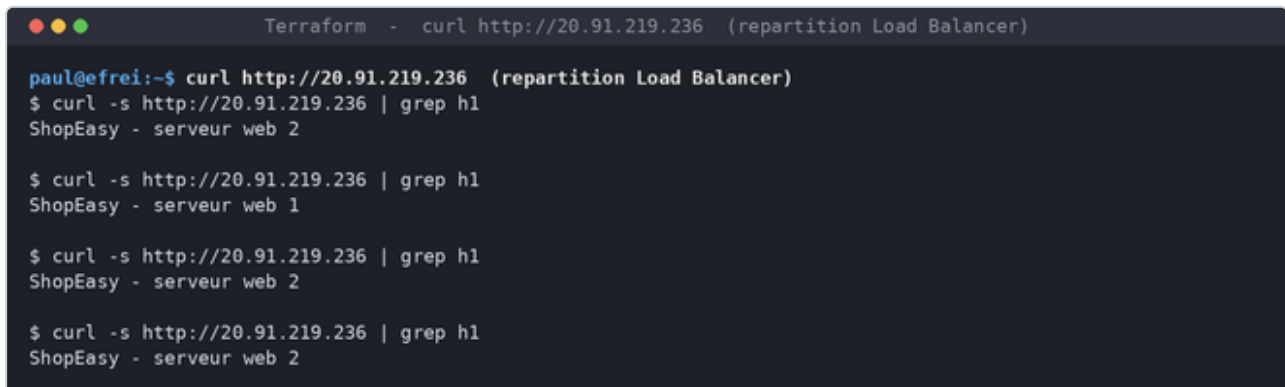
Envoyer des commentaires



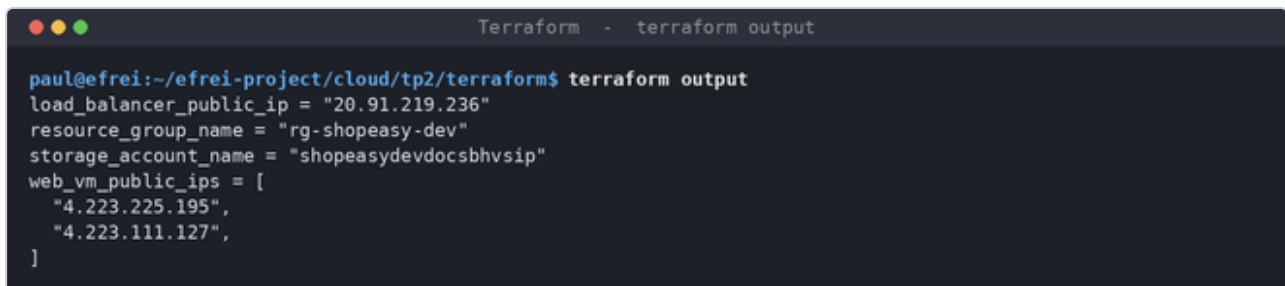
## atelier\_08-storage.png



## atelier\_09-curl-lb.png



## atelier\_09-output.png



## atelier\_09-page-lb.png

### ShopEasy - serveur web 2

Deploiement realise avec Terraform sur Azure.

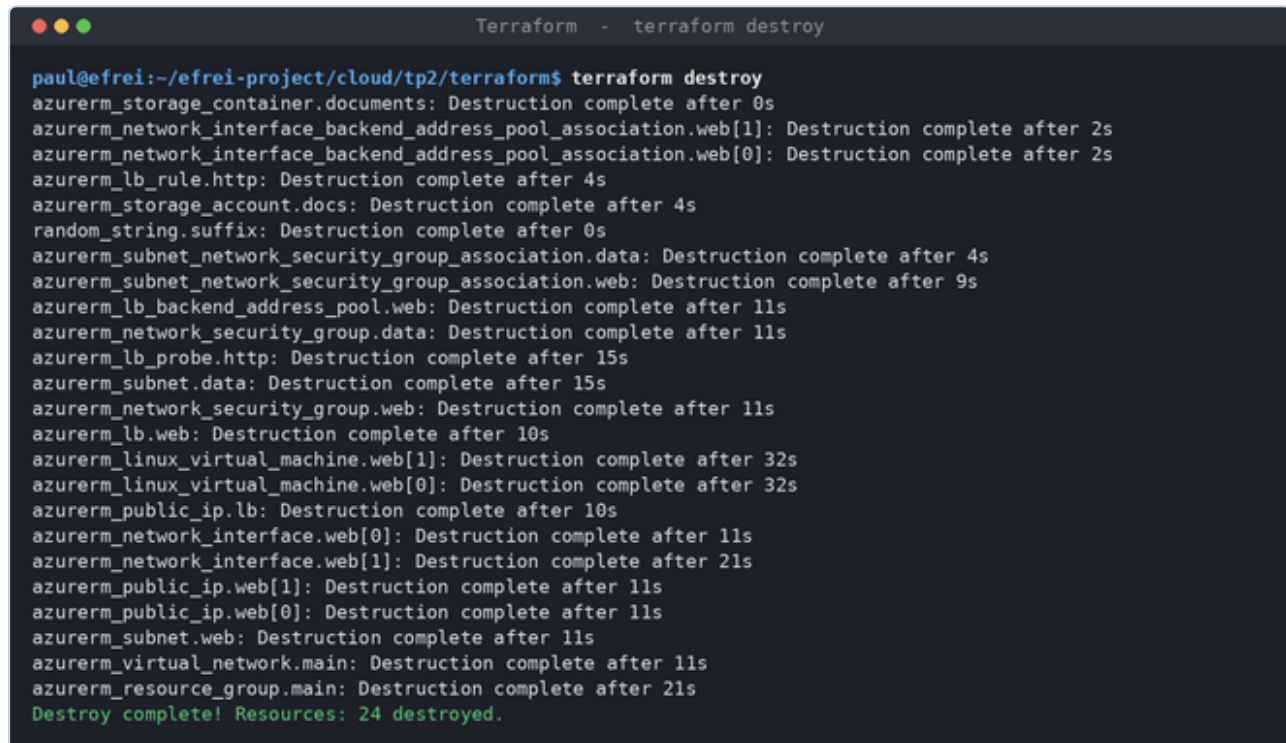
## atelier\_11-drift-plan.png

```
Terraform - detection de derive (drift)

paul@efrei:~/.../tp2/terraform$ detection de derive (drift)
$ az tag update ... --tags manual_change=true # modification manuelle hors Terraform
$ terraform plan

# azurerm_resource_group.main will be updated in-place
- resource "azurerm_resource_group" "main" {
  id      = "/subscriptions/cdca6d99-645a-4251-85e1-f078d1bd66ff/resourceGroups/rg-shopeasy-dev"
  name    = "rg-shopeasy-dev"
  - tags   = {
    "cost_center" = "cloud-training"
    "environment" = "dev"
    "managed_by"  = "terraform"
    - "manual_change" = "true" -> null
    "owner"       = "formation"
    "project"     = "shopeasy"
  }
}
Plan: 0 to add, 1 to change, 0 to destroy.
```

## atelier\_14-destroy.png



```
Terraform - terraform destroy

paul@efrei:~/efrei-project/cloud/tp2/terraform$ terraform destroy
azurerm_storage_container.documents: Destruction complete after 0s
azurerm_network_interface_backend_address_pool_association.web[1]: Destruction complete after 2s
azurerm_network_interface_backend_address_pool_association.web[0]: Destruction complete after 2s
azurerm_lb_rule.http: Destruction complete after 4s
azurerm_storage_account.docs: Destruction complete after 4s
random_string.suffix: Destruction complete after 0s
azurerm_subnet_network_security_group_association.data: Destruction complete after 4s
azurerm_subnet_network_security_group_association.web: Destruction complete after 9s
azurerm_lb_backend_address_pool.web: Destruction complete after 11s
azurerm_network_security_group.data: Destruction complete after 11s
azurerm_lb_probe.http: Destruction complete after 15s
azurerm_subnet.data: Destruction complete after 15s
azurerm_network_security_group.web: Destruction complete after 11s
azurerm_lb.web: Destruction complete after 10s
azurerm_linux_virtual_machine.web[1]: Destruction complete after 32s
azurerm_linux_virtual_machine.web[0]: Destruction complete after 32s
azurerm_public_ip.lb: Destruction complete after 10s
azurerm_network_interface.web[0]: Destruction complete after 11s
azurerm_network_interface.web[1]: Destruction complete after 21s
azurerm_public_ip.web[1]: Destruction complete after 11s
azurerm_public_ip.web[0]: Destruction complete after 11s
azurerm_subnet.web: Destruction complete after 11s
azurerm_virtual_network.main: Destruction complete after 11s
azurerm_resource_group.main: Destruction complete after 21s
Destroy complete! Resources: 24 destroyed.
```